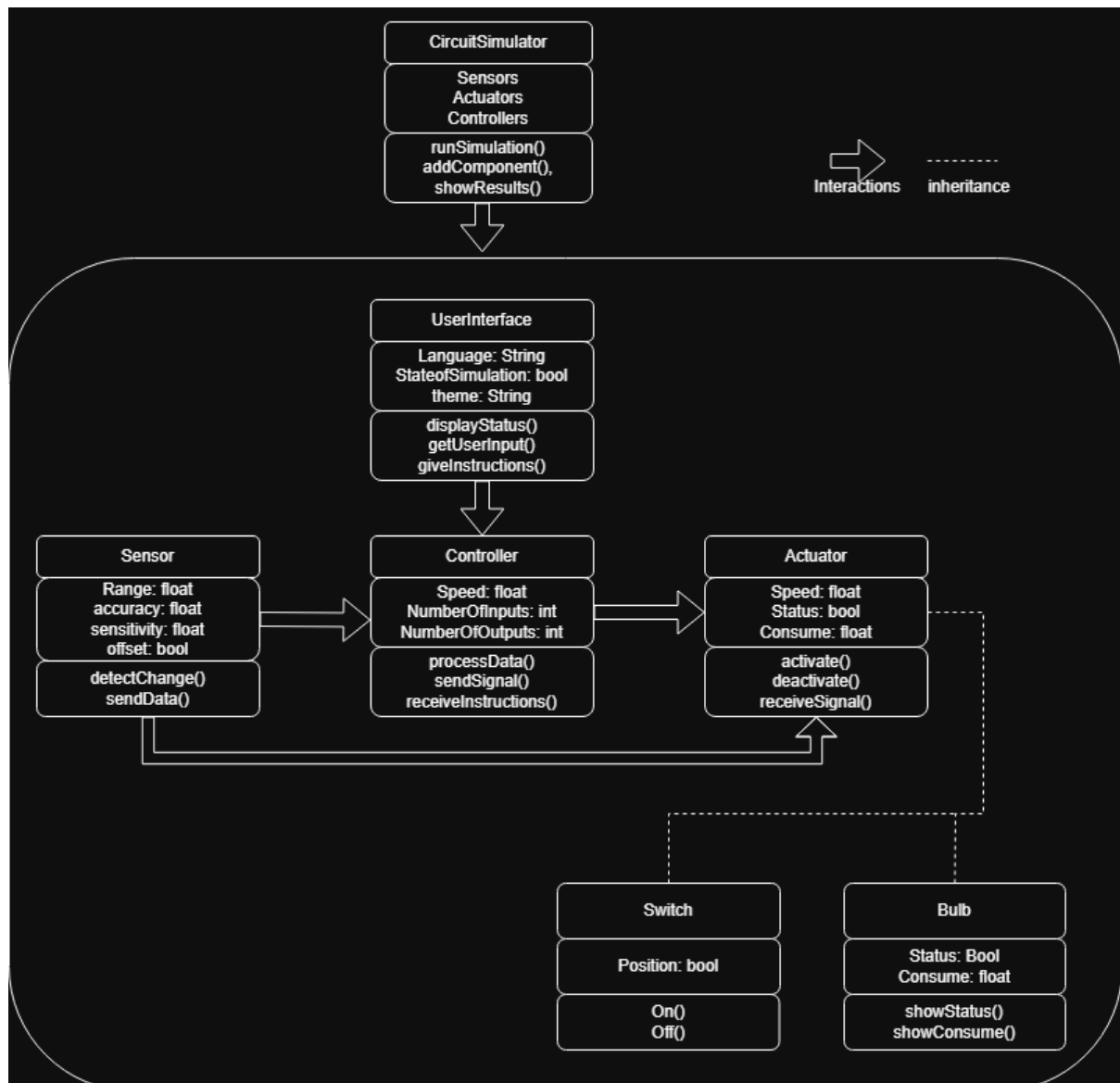


Workshop No. 2 — Object-Oriented Implementation for a Domotic Circuit Simulator

Andres David Hincapié Álvarez
Samuel Alejandro Bulla Fiquitiva
Daniel Alejandro Clavijo Gonzalez

Conceptual Design Updates:

Technical Design (UML Diagrams):



The diagram structures the system into classes that represent components of the circuit simulator. Inheritance is used in classes such as **Switch** and **Bulb**, which derive from **Actuator** and override methods like `activate()` for specific behaviors. Polymorphism allows different actuators to be handled in a generic way. Moreover, **abstraction** is evident in classes like **CircuitSimulator** and **UserInterface**, which manage the simulation and user interaction without exposing internal details.

Implementation Plan for OOP Concepts:

The system we want to create in our application will be based on the UML diagram — everything will originate from (**CircuitSimulator**), from which different components such as Sensor, Controller, Actuator, and UserInterface will branch out. The goal is for it to be scalable and easy to maintain.

Encapsulation:

This will be used to protect each class, exposing only the necessary methods and preventing direct modification of the main class. For example:

The Sensor will store attributes such as **range**, **accuracy**, **sensitivity**, and **offset**, and provide public methods like **detectChange()** and **sendData()** to send information to the controller.

The Controller will receive data from sensors through methods like **processData()** and **receiveInstructions()**, without allowing direct modification of its internal attributes (**numberOfInputs**, **numberOfOutputs**).

The Actuator will keep its attributes (**speed**, **status**, **consume**) encapsulated, managing its state through the methods **activate()**, **deactivate()**, and **receiveSignal()**.

The UserInterface will handle the system's visual state (**language**, **theme**, **simulationstatus**) through public methods like **displayStatus()** and **getUserInput()**.

Inheritance:

The design establishes a hierarchy among actuators — the Actuator class serves as a base for defining others.

Child classes:

Switch

Bulb

These inherit the main attributes and methods from Actuator but implement specific behaviors:

Switch controls its position (**On()** and **Off()**), while Bulb displays its state and consumption (**showStatus()** and **showConsume()**).

If new actuators like **Fan** or **DoorLock** are added in the future, they can also derive from **Actuator** without modifying existing code.

Polymorphism:

The simulator will leverage polymorphism to interact with different objects in a generic way:

The Controller will handle a list of actuators (**Actuator**) and send them signals without knowing their specific type.

Each class derived from **Actuator** will override the **receiveSignal()** method to execute its particular behavior.

Similarly, all Sensors will implement the **sendData()** method, allowing the controller to process them within the same iteration.

Initial Python Code Snippets:

The **Home Automation Circuit Simulator** was developed using **object-oriented programming** principles and a **modular layered architecture** to keep the system organized, scalable, and easy to understand.

- **main.py** serves as the program's entry point, integrating the three layers —**presentation, logic, and data**— by connecting the **DataManager**, **CircuitSimulator**, and **UserInterface**.
- In the **logic layer**, classes define the internal behavior of the simulator.
 - **Component** is the base class.
 - **Bulb**, **Switch**, **Sensor**, **Actuator**, and **Controller** extend its functionality to represent circuit elements and their interactions.
 - **CircuitSimulator** manages all components and runs the overall simulation.
- The **data layer**, managed by **DataManager**, saves and retrieves circuit information in **JSON** files, allowing users to continue their work across sessions.
- The **presentation layer**, implemented with **PyQt5** in the **UserInterface** class, provides a graphical interface with buttons to add, connect, simulate, save, and load components without using the console.

Overall, the system applies **encapsulation, modularity, and single responsibility principles**, resulting in a **functional, clear, and maintainable simulator** where each layer works **independently yet cooperatively**.

<https://github.com/danielclavijo23/CLASE-POO>